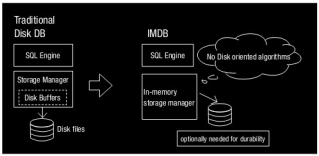

Figure 1: IMDB vs Disk DB

were built and maintained as internal components. IBM built one of the first in-memory engines (IMS/VS FastPath) way back in 1978. The first significant commercial IMDB offering was TimesTen (later acquired by Oracle). Since then, almost all top DB vendors boast an IMDB product—like IBM's SolidDB, Sybase's ASE, etc.

Myths about IMDBs

Consider the following assertions:

- Given the same amount of RAM, disk DBs can perform at the same speed as IMDBs (by using caching technology).
- If a RAM disk is created and a traditional disk DB is deployed on it, it delivers the same performance as an in-memory database.
- If the system crashes, all the data stored in IMDBs will be lost.
- SSDs and flash storage are getting better and better. Using these technologies along with traditional disk DBs yields the same performance as an in-memory database.
- Since RAM size is limited, sizes of IMDBs are also limited.
- Finally, IMDBs are not special; they are traditional disk DBs made to run on RAM.

Naturally, all of these are myths. It is close to impossible to create an in-memory DB from a traditional disk database by just changing the OS or hardware environment.

So, internally, how different is an IMDB from a traditional disk DB?

IMDBs are architected and designed keeping in mind the fact that all data is in memory. This actually leads to much simpler design as compared to disk DBs. There are six areas of difference:

1. *Query optimisation:* In disk DBs, the I/O cost factor dominates the optimisation. However, in IMDBs there is no such clear factor, which makes query optimisation very tricky. This is generally solved by taking constants and falling back on rule-based optimisation.
2. *Indexing:* More memory-friendly data structures and algorithms are used for indexing. While most disk DBs use B-Tree as a primary indexing data structure/algorithm, IMDBs tend to use T-Tree as a primary indexing data structure/algorithm.
3. *Internal data representation:* Compactness of representation dominates concerns for IMDBs. With all data being in memory, IMDBs tend to use direct memory pointers heavily. This is very typical of the IMDB memory page, index data or relation representations.
4. *Durability and recovery:* Contrary to popular belief, IMDBs are durable. They use algorithms similar to disk DBs for persistence. However, the buffer management, which is the biggest performance bottleneck for disk DBs, is eliminated.

   During database loading, IMDBs tend to take a bit more time as they have to load the complete data into memory. Hence, recovery is a bit slower.
5. *Access methodology:* Generally, disk DBs offer client server over sockets as a primary access method. However, with no disk I/O, if IMDBs only offer sockets for access, this will become a bottleneck. Hence, most IMDBs tend to offer shared-memory access as a primary method. In a few cases, JDBC/ODBC interfaces are also supported.
6. *Concurrency control:* Due to inherent speed in processing, IMDBs can take coarser locks and also do less to persist them. However, disk DBs take finer locks and take elaborate measures to persist them.

   Figure 2 is a typical architecture for an in-memory database.

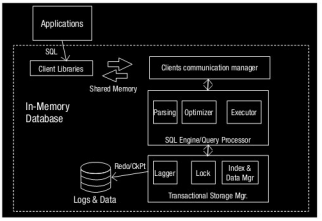

Figure 2: IMDB architecture

Typical applications of IMDBs

IMDBs are applicable in all domains that require real-time performance and very low latency. Four domains typically use IMDBs: telecom, financial segments, enterprises, and e-commerce and Web applications. In these spaces, IMDBs are used in a variety of applications (refer to Figure 3).

Key offerings in the IMDB space

The IMDB space is dominated by a lot of commercial players. Some of the most important ones are Oracle TimesTen, IBM SolidDB, Sybase ASE, ENEA Polyhedra and McObject ExtremeDB. There are also some typical open source solutions (refer to Figure 4).

Let us take a look at two of the FOSS IMDBs, CSQL and MonetDB.

CSQL: CSQL is an open source main-memory high-performance RDBMS developed in India. It is one of the